

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

I. Introduction

The goal of this lab project is the design and implementation of a Finite-State Machine through the form of an Up-Down Counter in Xilinx Verilog code. This finite-state machine in particular will cycle through nine states which represent the nine digits of my RedID. The circuit simulated is that of a Moore circuit, where the output Z will be determined solely by the state, which is deemed to be simpler in representing repeating digits in the case of my particular RedID. The testbench will demonstrate two inputs, the counter counting up, as well as the counter counting down, showcasing the cycling and repeat of the nine digits.

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

II. Theoretical Preparation

Designing an Up-Down Counter

Table 1: Next State Table

Current State	Next State - Q1+,Q2+,Q3+,Q4+				Output
Q1,Q2,Q3,Q4	UD 00	UD 01	UD 10	UD 11	Z1,Z2,Z3,Z4
S0 - 0000	0000	1000	0001	0000	1 - 0001
S1 - 0001	0001	0000	0010	0001	3 - 0011
S2 - 0010	0010	0001	0011	0010	0 - 0000
S3 - 0011	0011	0010	0100	0011	5 - 0101
S4 - 0100	0100	0011	0101	0100	5 - 0101
S5 - 0101	0101	0100	0110	0101	1 - 0001
S6 - 0110	0110	0101	0111	0110	5 - 0101
S7 - 0111	0111	0110	1000	0111	7 - 0111
S8 - 1000	1000	0111	0000	1000	4 - 0100

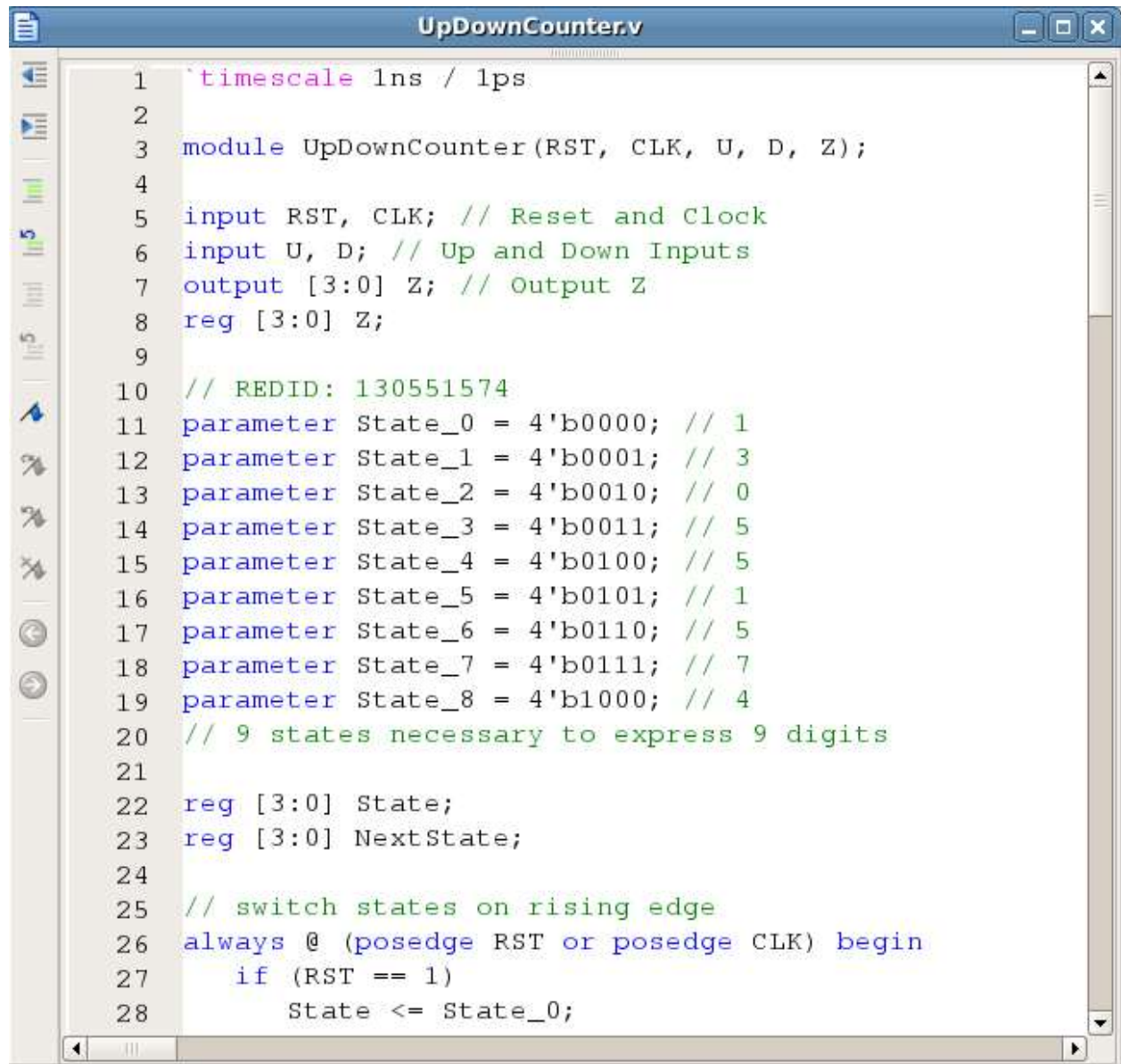
First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

III. Experimental Setup

Verilog Code for Up-Down Counter

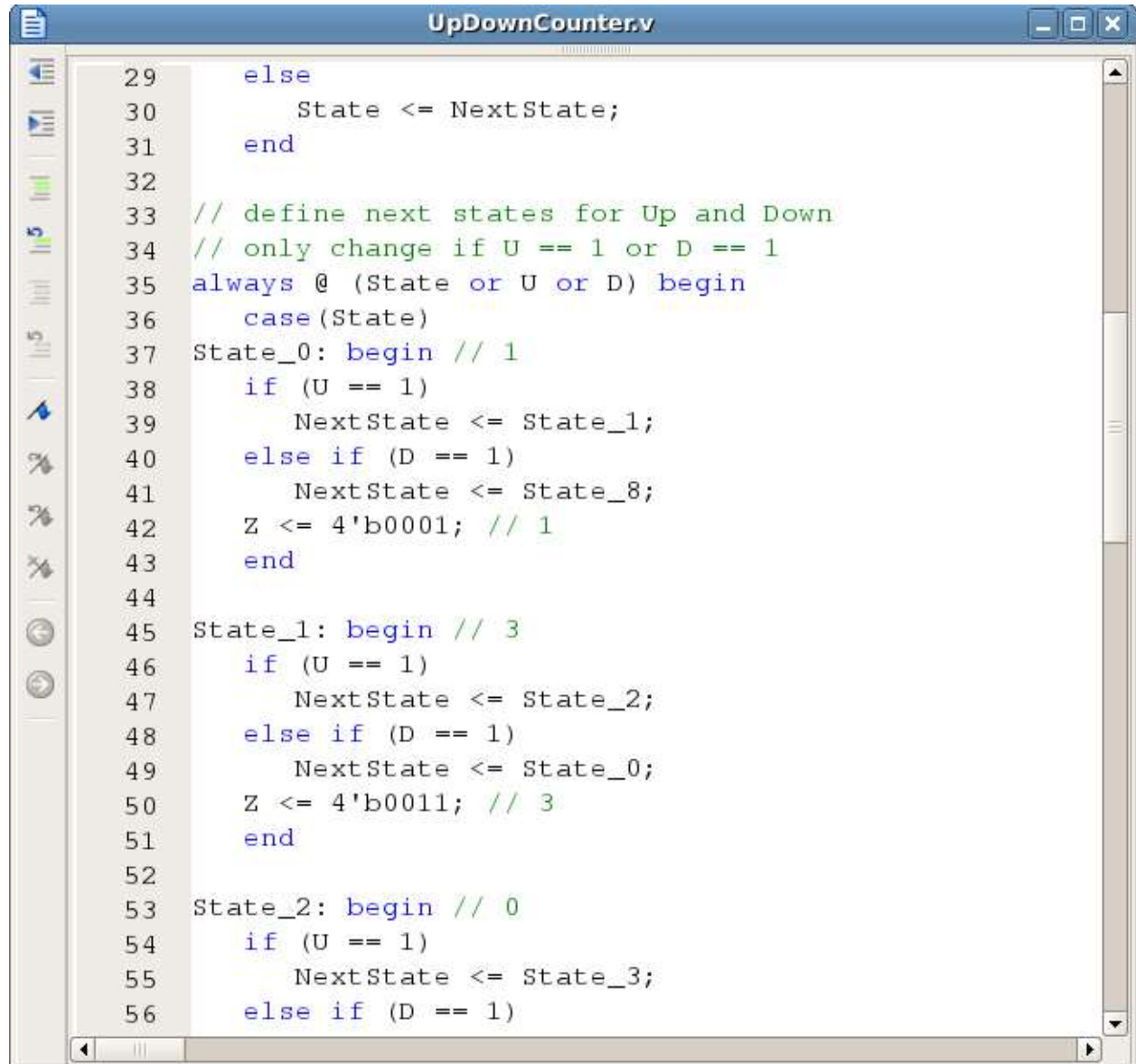


```
1  `timescale 1ns / 1ps
2
3  module UpDownCounter(RST, CLK, U, D, Z);
4
5  input RST, CLK; // Reset and Clock
6  input U, D; // Up and Down Inputs
7  output [3:0] Z; // Output Z
8  reg [3:0] Z;
9
10 // REDID: 130551574
11 parameter State_0 = 4'b0000; // 1
12 parameter State_1 = 4'b0001; // 3
13 parameter State_2 = 4'b0010; // 0
14 parameter State_3 = 4'b0011; // 5
15 parameter State_4 = 4'b0100; // 5
16 parameter State_5 = 4'b0101; // 1
17 parameter State_6 = 4'b0110; // 5
18 parameter State_7 = 4'b0111; // 7
19 parameter State_8 = 4'b1000; // 4
20 // 9 states necessary to express 9 digits
21
22 reg [3:0] State;
23 reg [3:0] NextState;
24
25 // switch states on rising edge
26 always @ (posedge RST or posedge CLK) begin
27     if (RST == 1)
28         State <= State_0;
```

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

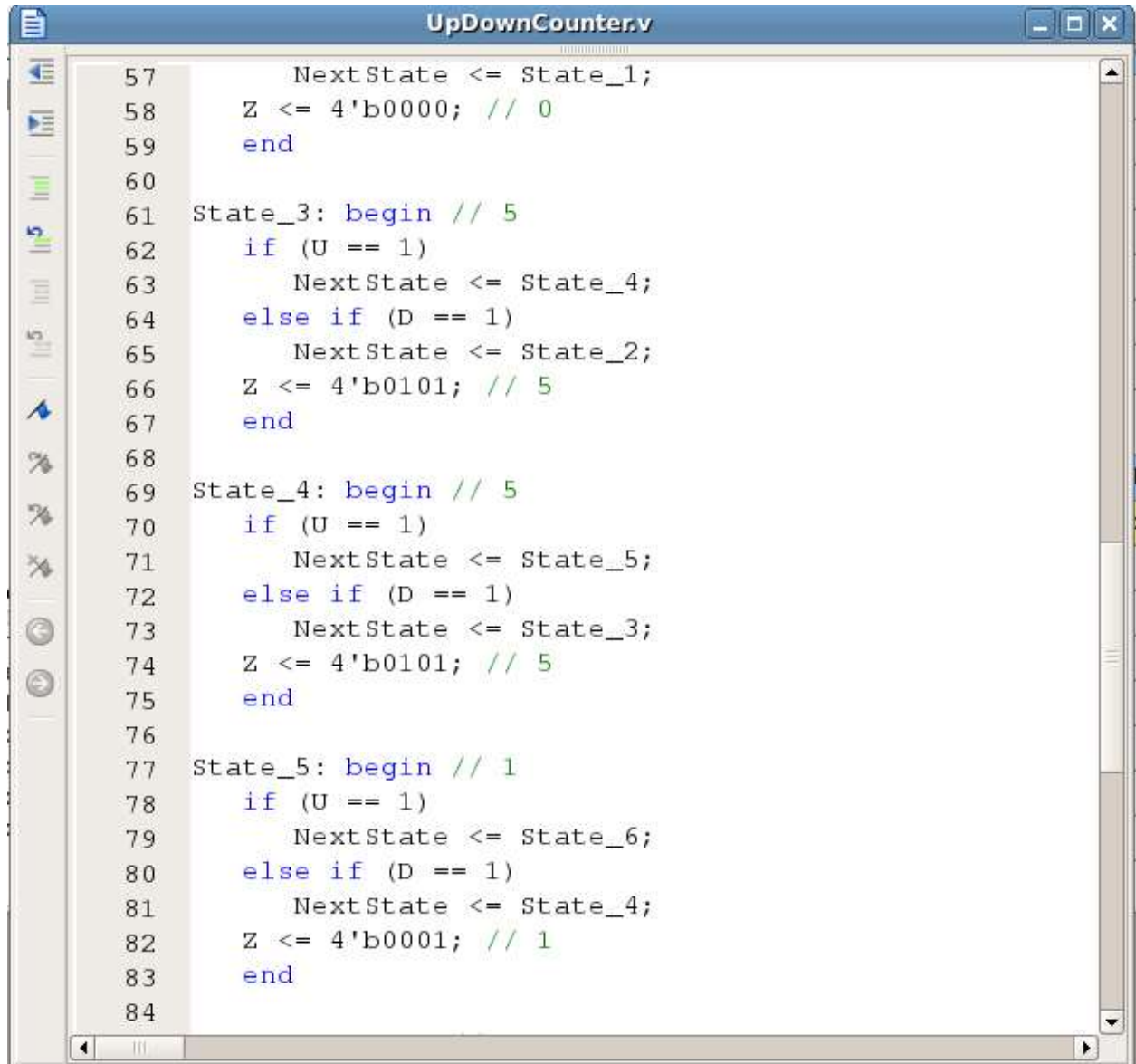


```
29     else
30         State <= NextState;
31     end
32
33     // define next states for Up and Down
34     // only change if U == 1 or D == 1
35     always @ (State or U or D) begin
36         case(State)
37     State_0: begin // 1
38             if (U == 1)
39                 NextState <= State_1;
40             else if (D == 1)
41                 NextState <= State_8;
42             Z <= 4'b0001; // 1
43         end
44
45     State_1: begin // 3
46             if (U == 1)
47                 NextState <= State_2;
48             else if (D == 1)
49                 NextState <= State_0;
50             Z <= 4'b0011; // 3
51         end
52
53     State_2: begin // 0
54             if (U == 1)
55                 NextState <= State_3;
56             else if (D == 1)
```

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

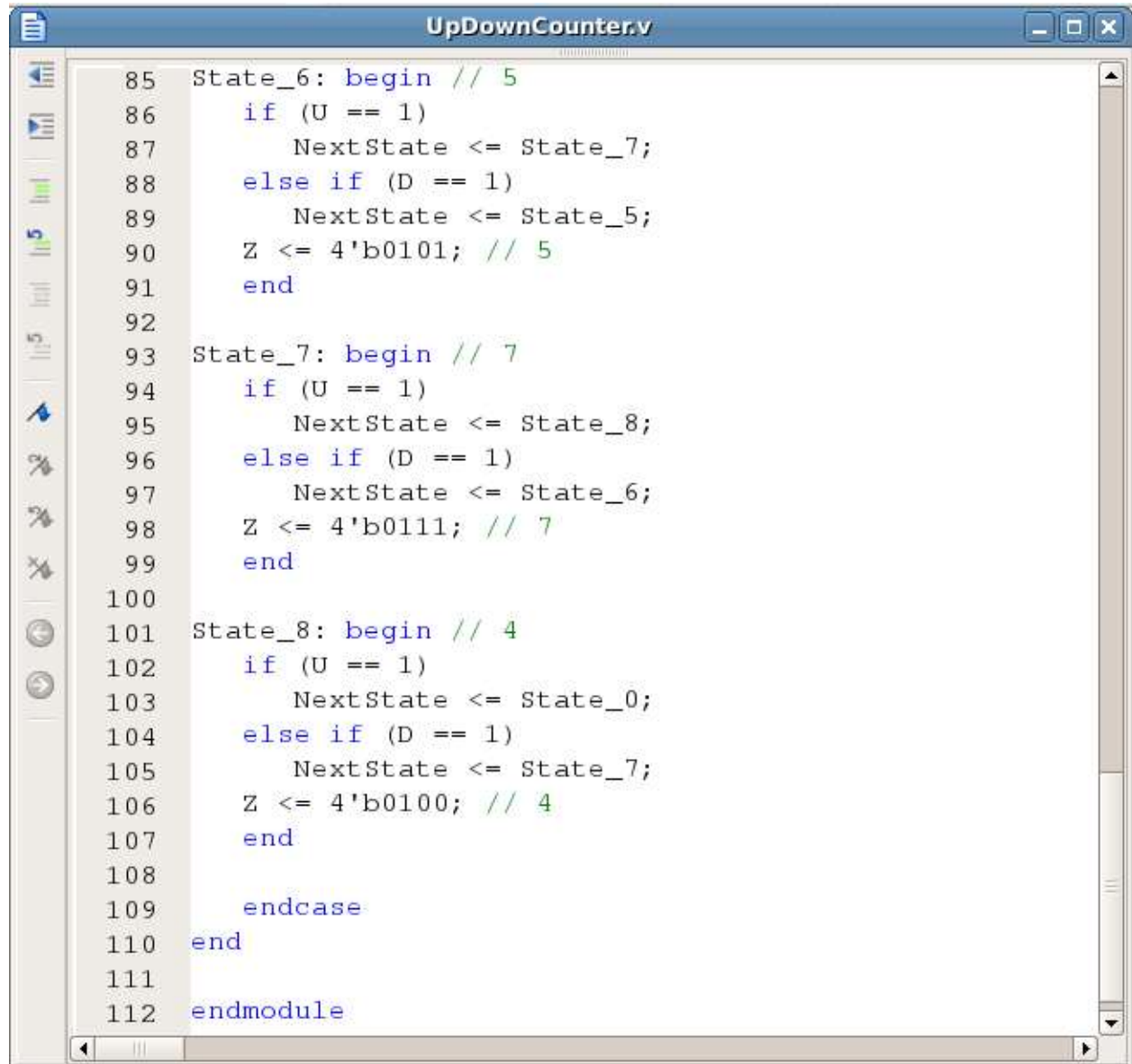


```
57     nextState <= State_1;
58     Z <= 4'b0000; // 0
59     end
60
61 State_3: begin // 5
62     if (U == 1)
63         nextState <= State_4;
64     else if (D == 1)
65         nextState <= State_2;
66     Z <= 4'b0101; // 5
67     end
68
69 State_4: begin // 5
70     if (U == 1)
71         nextState <= State_5;
72     else if (D == 1)
73         nextState <= State_3;
74     Z <= 4'b0101; // 5
75     end
76
77 State_5: begin // 1
78     if (U == 1)
79         nextState <= State_6;
80     else if (D == 1)
81         nextState <= State_4;
82     Z <= 4'b0001; // 1
83     end
84
```

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574



```
85 State_6: begin // 5
86     if (U == 1)
87         NextState <= State_7;
88     else if (D == 1)
89         NextState <= State_5;
90     Z <= 4'b0101; // 5
91 end
92
93 State_7: begin // 7
94     if (U == 1)
95         NextState <= State_8;
96     else if (D == 1)
97         NextState <= State_6;
98     Z <= 4'b0111; // 7
99 end
100
101 State_8: begin // 4
102     if (U == 1)
103         NextState <= State_0;
104     else if (D == 1)
105         NextState <= State_7;
106     Z <= 4'b0100; // 4
107 end
108
109 endcase
110 end
111
112 endmodule
```

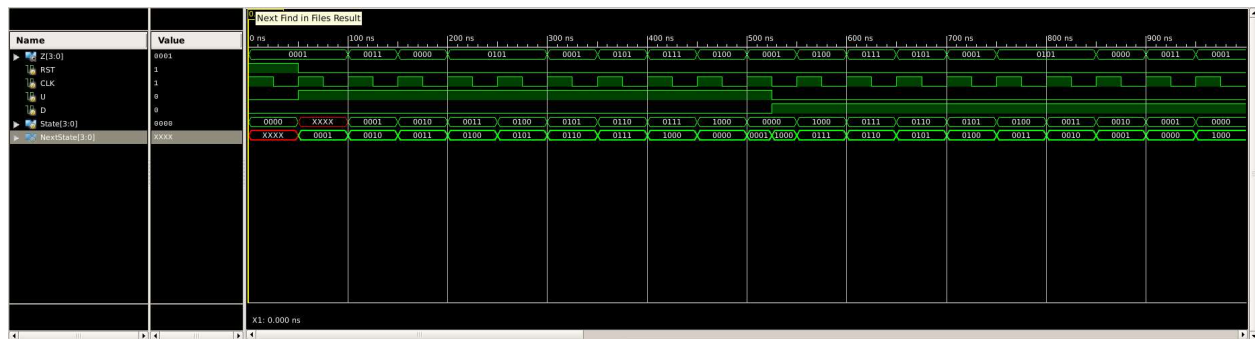
First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

IV. Experimental Result

Waveform for Up-Down Counter



Code assigns input variables U and D to both 0 and 0 (UD 00), and then sets U to 1 (UD 10) for 10 full clock-cycles, and then simultaneously assigns U to 0 and D to 1 (UD 01) for 10 full clock-cycles. This was done to show how the Z output will eventually cycle back to the first output 0001 (1) and that the D input eventually does the same as well. The output is read at each rising edge of the clock, so sequential digits that have the same value (i.e the 55 of the 130551574 sequence) are represented by a signal held for two rising edges.

First Name: Rhilo Novuno

Last Name: Sotto

RedID: 130551574

V. Conclusion

In conclusion, this project has implemented both a Finite-State Machine and a RedID counter in Verilog code. The implementation and design were based on the principles of sequential circuits derived from the next-state table of the up-down counter, with the code showcasing the sequence of outputs given the inputs. Through this lab project, experience has been gained in utilizing the Xilinx ISE as well as Verilog code to create more complex systems higher than the logic-gate level. Overall, this demonstrates an understanding of sequential circuit design principles such as using next-state tables, and Moore vs. Mealy circuits.
